UF2405 | UNIDAD DIDÁCTICA 3 MANUAL TÉCNICO DEL DESARROLLADOR

Arquitectura, componentes, compilación y mantenimiento

Proyecto	actividad3-uf2405
Autor	Sara Lopez Figueredo Sainz
Versión	1.0
Fecha	Octubre de 2026

Documento de apoyo para la entrega de la Actividad 3.

Índice

1. Introducción y alcance

2. Arquitectura

3. Tecnologías

4. Estructura del proyecto

5. Componentes principales

6. Configuración y compilación

7. Flujo de una petición HTTP

8. Guía de ampliación

9. Patrones y principios

10. Despliegue

11. Seguridad y mantenimiento

12. Diagnóstico de errores

13. Referencias

14. Historial de cambios

1. Introducción y alcance

Este manual está dirigido a desarrolladores, personal técnico y estudiantes que necesiten comprender, mantener o ampliar actividad3-uf2405. Describe la estructura declarada del proyecto y las instrucciones de compilación y despliegue recogidas en la documentación de la actividad.

1.1. Objetivos

Documentar la arquitectura y los componentes principales.

Explicar la relación entre Java Servlet, JSP, CSS y JavaScript.

Facilitar la compilación y el arranque local.

Proporcionar pautas para extender la aplicación y diagnosticar errores.

1.2. Alcance y cautelas

La aplicación es un ejemplo educativo de presentación web con una ruta de entrada Servlet y una vista JSP. No se describe una capa de modelo persistente ni una base de datos. Antes de considerar el proyecto apto para producción, deben verificarse las versiones reales, el archivo pom.xml, la configuración del contenedor y las medidas de seguridad.

2. Arquitectura

La solución separa el contenido y la presentación del comportamiento en cliente, y utiliza un Servlet como punto de entrada y una JSP como vista.

Capa o componente	Archivo/tecnología	Responsabilidad
Presentación	index.jsp + HTML	Estructura y contenido que recibe el usuario.
Estilos	css/estilos.css	Colores, tipografía, márgenes y adaptación visual.
Interactividad	js/script.js	Eventos del botón y mensajes en el navegador.
Controlador	Actividad3Servlet.java	Procesa la petición y deriva el control a la vista.
Servidor	Apache Tomcat	Contenedor de la aplicación web y procesamiento de JSP/Servlet.
Construcción	Apache Maven	Gestión de dependencias y tareas de compilación.

2.1. Recorrido general

Navegador → Tomcat → configuración de bienvenida → Actividad3Servlet.doGet()\nActividad3Servlet → forward a index.jsp → motor JSP → HTML generado → navegador\nNavegador → ejecución de css/estilos.css y js/script.js

2.2. Principios de diseño

Separación de responsabilidades: estructura, estilos y comportamiento se mantienen en archivos diferentes.

Portabilidad: los scripts pretenden utilizar herramientas locales dentro del proyecto.

Mantenibilidad: la organización por carpetas facilita localizar cada componente.

Ausencia de persistencia descrita: la aplicación de ejemplo no requiere una base de datos.

3. Tecnologías

Componente	Tecnología/versión indicada	Uso
Lenguaje del servidor	Java, JDK 8 o superior	Implementación del Servlet.
Servidor	Apache Tomcat 7.0.47	Contenedor web indicado en la documentación.
Gestión de dependencias	Apache Maven 3.9.9	Compilación y ejecución del objetivo configurado.
API de Servlet	javax.servlet-api 3.1.0	Tipos y contratos de Servlet.
Vista	JSP 2.3 / HTML5	Generación y estructura del contenido.
Estilos	CSS3	Diseño visual.
Cliente	JavaScript ES5	Interacción en el navegador.

Compatibilidad: Tomcat 7 es una versión antigua y fuera de soporte. La combinación exacta de Tomcat, Java y plugins debe comprobarse en el entorno real. Este manual conserva las versiones indicadas por el material de partida y no certifica su compatibilidad actual.

4. Estructura del proyecto

actividad3-uf2405/\n ├— pom.xml\n ├— instalar_maven_local.bat\n ├— lanzar_proyecto.bat\n ├— comprimir_proyecto.bat\n ├— README.md\n ├— doc/\n│ ├— 01_memoria_tecnica.md\n│ ├— 02_bitacora_errores.md\n│ ├— 03_aprendizajes.md\n│ └— 04_guia_despliegue.md\n ├— tools/\n│ └— apache-maven-3.9.9/\n└— src/main/\n ├— java/com/empresa/web/Actividad3Servlet.java\n

└── webapp/\n ├── WEB-INF/web.xml\n ├── css/estilos.css\n ├── js/script.js\n └── index.jsp

Ruta	Contenido
src/main/java/	Código fuente Java.
src/main/webapp/	Vistas JSP y recursos web.
src/main/webapp/WEB-INF/	Configuración protegida de la aplicación.
tools/	Herramientas locales descargadas por los scripts.
target/	Artefactos generados durante la compilación; normalmente no se versionan.
doc/	Documentación técnica y de apoyo.

5. Componentes principales

5.1. pom.xml

Se ubica en la raíz y define las coordenadas del proyecto, el empaquetado WAR, las dependencias y los plugins de compilación.

<groupId>com.empresa</groupId>\n<artifactId>actividad3-uf2405</artifactId>\n<version>1.0-SNAPSHOT</version>\n<packaging>war</packaging>

javax.servlet-api con alcance provided: la proporciona el contenedor Servlet.

maven-compiler-plugin: configura la compilación Java.

tomcat7-maven-plugin: se indica para ejecutar la aplicación en el entorno de desarrollo.

5.2. Actividad3Servlet.java

Ruta: src/main/java/com/empresa/web/Actividad3Servlet.java. El Servlet extiende HttpServlet y deriva las peticiones GET y POST a index.jsp.

package com.empresa.web;\n\nimport javax.servlet.ServletException;\nimport javax.servlet.annotation.WebServlet;\nimport javax.servlet.http.HttpServlet;\nimport javax.servlet.http.HttpServletRequest;\nimport javax.servlet.http.HttpServletResponse;\nimport java.io.IOException;\n\n@WebServlet(\"/Actividad3Servlet\")\npublic class Actividad3Servlet extends HttpServlet {\n @Override\n protected void doGet(HttpServletRequest request,\n HttpServletResponse response)\n throws ServletException, IOException {\n request.getRequestDispatcher(\"index.jsp\")\n .forward(request, response);\n }\n\n @Override\n protected void doPost(HttpServletRequest request,\n HttpServletResponse response)\n throws ServletException, IOException {\n doGet(request, response);\n }\n}

Conceptos de POO: herencia (extiende HttpServlet), polimorfismo (sobrescribe doGet y doPost), encapsulamiento (métodos protected) y abstracción (utiliza la API del contenedor para tratar peticiones HTTP).

5.3. index.jsp

Ruta: src/main/webapp/index.jsp. Presenta el contenido HTML e incluye los recursos CSS y JavaScript. La documentación indica que puede generar una fecha de servidor con una expresión JSP.

<%\n   String fechaServidor = new java.util.Date().toString();\n%>\n Fecha: <%= fechaServidor %>

Conviene mantener la lógica de negocio fuera de la JSP y limitarla a la presentación. La fecha solo aparecerá si el fragmento está realmente integrado en la versión entregada.

5.4. estilos.css

Ruta: src/main/webapp/css/estilos.css. Define el fondo #FFFF00, la tipografía Times New Roman, la imagen centrada, los separadores azules y los estilos del botón, incluido el estado hover.

5.5. script.js

Ruta: src/main/webapp/js/script.js. Contiene la función mostrarMensaje(), que presenta la alerta solicitada, y puede incluir registros de diagnóstico en la consola.

function mostrarMensaje() {\n   alert("Mensaje mostrado al pulsar el botón");\n console.log("Botón pulsado. Mensaje mostrado al usuario.");\n}\n\nwindow.onload = function() {\n   console.log("JavaScript cargado correctamente.");\n   console.log("Fecha de carga: " + new Date().toLocaleString());\n};

5.6. web.xml

Ruta: src/main/webapp/WEB-INF/web.xml. La documentación indica la configuración de una página de bienvenida que apunta al Servlet:

<welcome-file-list>\n   <welcome-file>Actividad3Servlet</welcome-file>\n</welcome-file-list>

Debe comprobarse que la configuración del descriptor y el mapeo del Servlet coinciden con la ruta de despliegue.

6. Configuración y compilación

6.1. Requisitos previos

JDK compatible instalado y accesible desde PATH.

Editor de código, por ejemplo Visual Studio Code, IntelliJ IDEA o Eclipse.

Navegador con herramientas de desarrollo.

6.2. Maven local

El script instalar_maven_local.bat pretende crear tools/, descargar Maven 3.9.9 y extraerlo en tools/apache-maven-3.9.9/. Revise el resultado antes de continuar.

6.3. Arranque de desarrollo

El script lanzar_proyecto.bat comprueba Java, localiza Maven y ejecuta el objetivo configurado. El comando documentado es:

"%MAVEN_CMD%" clean compile tomcat7:run

La disponibilidad de este objetivo depende de la configuración efectiva del pom.xml y de la compatibilidad de las versiones.

7. Flujo de una petición HTTP

El usuario abre http://localhost:8080/actividad3/ en el navegador.

Tomcat recibe la petición GET y consulta la configuración de bienvenida.

El contenedor resuelve la ruta al Servlet configurado.

Actividad3Servlet.doGet() deriva la petición a index.jsp mediante forward().

El motor JSP procesa las expresiones dinámicas y genera HTML.

Tomcat devuelve la respuesta al navegador.

El navegador interpreta el HTML, aplica CSS y ejecuta JavaScript.

7.1. Ciclo de vida del Servlet

init(): inicialización del Servlet por el contenedor.

doGet(): tratamiento de peticiones HTTP GET.

doPost(): tratamiento de peticiones HTTP POST; en el ejemplo delega en doGet().

destroy(): liberación de recursos al retirar el Servlet o detener el contenedor.

8. Guía de ampliación

8.1. Añadir una función JavaScript

Añada la función en js/script.js y vincúlela a un control de index.jsp.

function nuevaFuncionalidad() {\n console.log("Nueva funcionalidad ejecutada");\n}\n\n// Ejemplo de integración en la vista:\n<button onclick="nuevaFuncionalidad()">Nuevo botón</button>

8.2. Modificar estilos

Elemento	Valor documentado
Fondo	#FFFF00
Botón y separadores	#4A90E2
Hover del botón	#357ABD
Tipografía	Times New Roman, serif

8.3. Añadir un endpoint

Para incorporar una ruta nueva, cree un Servlet con un mapeo propio y gestione la petición en doGet() o doPost(). Evite duplicar lógica y valide cualquier dato que llegue desde el cliente.

@WebServlet("/nuevoEndpoint")\npublic class NuevoServlet extends HttpServlet {\n @Override\n protected void doGet(HttpServletRequest request,\n HttpServletResponse response)\n throws ServletException, IOException {\n response.setContentType("text/plain;charset=UTF-8");\n response.getWriter().write("Nuevo endpoint");\n }\n}

9. Patrones y principios

9.1. Modelo-Vista-Controlador (MVC)

Vista: index.jsp.

Controlador: Actividad3Servlet.java.

Modelo: no se identifica un modelo de datos persistente en la solución descrita.

9.2. Punto de entrada y separación de responsabilidades

El Servlet funciona como punto de entrada para la ruta configurada. HTML/JSP estructura y presenta el contenido; CSS controla su aspecto; JavaScript gestiona la interacción en el navegador. Dado que se trata de una aplicación sencilla, esta organización es una aproximación educativa a la separación por capas.

10. Despliegue

10.1. Generar un WAR

tools\apache-maven-3.9.9\bin\mvn.cmd clean package

El material de partida indica que el artefacto se genera en target/actividad3-uf2405.war. Confirme el nombre real en target/ tras la compilación.

10.2. Tomcat externo

Copie el WAR generado a la carpeta webapps/ de una instalación compatible de Tomcat.

Inicie el servidor y revise los registros de arranque.

Acceda a la URL correspondiente al nombre de contexto desplegado.

10.3. Docker (alternativa documentada)

FROM tomcat:7-jdk8\nCOPY target/actividad3-uf2405.war /usr/local/tomcat/webapps/\nEXPOSE 8080\n\n# Construcción y ejecución\ndocker build -t actividad3-uf2405 .\ndocker run -p 8080:8080 actividad3-uf2405

La imagen de Tomcat 7/JDK 8 se recoge como referencia del proyecto original, pero no se recomienda para producción por antigüedad y falta de soporte.

11. Seguridad y mantenimiento

La aplicación descrita es de solo lectura, no incluye autenticación, formularios ni base de datos. Estas características reducen el alcance de los riesgos, pero no sustituyen una revisión de seguridad.

Utilizar versiones de Java y Tomcat con soporte vigente en cualquier despliegue real.

Configurar HTTPS/TLS cuando la aplicación se publique fuera del equipo local.

Añadir cabeceras de seguridad adecuadas, por ejemplo X-Content-Type-Options y una política de framing.

Validar en el servidor cualquier dato recibido si se incorporan formularios.

Registrar errores relevantes sin exponer información sensible.

Revisar dependencias, permisos de scripts y configuración antes de publicar.

12. Diagnóstico de errores

Error	Causa posible	Acción recomendada
BUILD FAILURE	Error de compilación, paquete o importación.	Revise el primer error de Maven y compruebe package com.empresa.web y las rutas.
HTTP 404	Ruta incorrecta o aplicación no desplegada.	Compruebe el contexto de despliegue, el mapeo del Servlet y web.xml.
Puerto 8080 ocupado	Otro proceso escucha en ese puerto.	Ejecute netstat -ano | findstr :8080 y compruebe el PID antes de detener un proceso.
mvn no se reconoce	Maven no está en PATH.	Use la ruta de Maven local o revise instalar_maven_local.bat.
Recursos CSS/JS ausentes	Ruta incorrecta o caché del navegador.	Compruebe las rutas y recargue con Ctrl + F5.

13. Referencias

UF2405 — Modelo de programación web y bases de datos; certificado de profesionalidad IFCD0112.

Java Servlet Specification: https://javaee.github.io/servlet-spec/

JSP Specification: https://javaee.github.io/jsp-spec/

Apache Maven: https://maven.apache.org/

Documentación de Apache Tomcat: https://tomcat.apache.org/

HTML: https://html.spec.whatwg.org/

MDN Web Docs: https://developer.mozilla.org/

14. Historial de cambios

Versión	Fecha	Autor	Descripción
1.0	Octubre de 2026	[Nombre y apellidos]	Primera versión revisada y estructurada.